

MA3627 Lecture 7 – Decision Trees and Ensemble Methods

Several of the models studied so far belong to the parametric family: linear and polynomial regression assume the relationship between features and output can be captured by a fixed functional form whose parameters are estimated from data. Decision trees, like k -NN from Lecture 5, take a non-parametric approach. They make no assumption about the global form of f^* ; instead, they recursively partition the feature space $\mathcal{X}$ into rectangular regions and assign a constant prediction to each. Trees are interpretable and require no feature scaling, but – as we will see by the end of Section 1 – they are also highly unstable: small changes to the training data can produce a completely different tree. That instability turns out to be exactly the property that makes *ensembles* of trees so effective, and the second half of this lecture develops the three main ensemble families built on top of it: bagging, Random Forests, and gradient boosting.

1 The Hypothesis Class and Recursive Partitioning

Recall $\mathcal{H}_{\text{tree},L}$, the class of decision trees with at most L leaves, introduced in Lecture 5 as a canonical non-parametric hypothesis class. Each element is defined by a partition of $\mathcal{X}$ into at most L rectangular regions, each obtained by a sequence of axis-aligned splits $x_j \leq \theta$ or $x_j > \theta$, together with a constant prediction c_ℓ assigned to each region. A tree with L leaves has $L - 1$ internal nodes; the depth is the length of the longest root-to-leaf path.

Finding the partition that globally minimises the empirical risk over $\mathcal{H}_{\text{tree},L}$ is NP-hard for $L \geq 3$. In practice, trees are built greedily.

Definition 1.1. Recursive binary splitting builds a tree as follows. Begin with all training data at a root node. At the current node with data $\mathcal{D}_t$, consider every candidate split (j, θ) and compute the impurity reduction $\Delta(j, \theta)$ (Section 2). Choose $(\hat{j}, \hat{\theta}) = \arg \max_{j, \theta} \Delta(j, \theta)$, partition $\mathcal{D}_t$ into the two children accordingly, and recurse on each child until a stopping criterion is met.

For a node with n_t observations and p features, the cost of finding the best split is $O(p n_t \log n_t)$, giving a total tree-building cost of $O(p n d \log n)$ for depth d – manageable for moderate n , p , and d .

2 Impurity Measures and the Splitting Criterion

Let R be a node with n_R points, of which $n_{R,c}$ belong to class c , so $\hat{p}_c = n_{R,c}/n_R$.

Definition 2.1. The **Gini impurity** of R is $G(R) = 1 - \sum_c \hat{p}_c^2$. The **entropy** of R is $H(R) = -\sum_c \hat{p}_c \log_2 \hat{p}_c$ (with $0 \log_2 0 = 0$). For regression, the impurity of a node is its mean squared error, $\text{MSE}(R) = n_R^{-1} \sum_{i \in R} (y_i - \bar{y}_R)^2$.

Both classification measures are zero at a pure node and maximised when all classes are equally represented; for binary classification, $G(1/2) = 1/2$ and $H(1/2) = 1$ bit, and in general $G(R) \leq H(R) \ln 2$ for all class distributions.

Definition 2.2. A split (j, θ) divides R into R_L and R_R with $n_L = |R_L|$. The **impurity reduction** is

$$\Delta(j, \theta) = I(R) - \frac{n_L}{n_R} I(R_L) - \frac{n_R - n_L}{n_R} I(R_R).$$

Proposition 2.3. *For Gini impurity or entropy, every split into two non-empty children satisfies $\Delta(j, \theta) \geq 0$, with equality if and only if R_L and R_R have the same class proportions as R (equivalently, as each other). For regression MSE, the corresponding impurity reduction is likewise non-negative, and is zero if and only if the two child means equal the parent mean.*

For Gini impurity and entropy, this follows from strict concavity on the probability simplex and Jensen's inequality: the parent's class-proportion vector is exactly the size-weighted average of the two children's, $\hat{\mathbf{p}}_R = (n_L/n_R)\hat{\mathbf{p}}_{R_L} + ((n_R - n_L)/n_R)\hat{\mathbf{p}}_{R_R}$, so Jensen's inequality gives $\Delta \geq 0$, with equality precisely when $\hat{\mathbf{p}}_{R_L} = \hat{\mathbf{p}}_{R_R}$ (and hence both equal $\hat{\mathbf{p}}_R$) by strict concavity.

For regression MSE, the result instead follows from the within-between sum-of-squares (ANOVA-style) decomposition. Write $w_L = n_L/n_R$ and $w_R = (n_R - n_L)/n_R$ for the proportion of the parent's observations falling in each child, and $\bar{y}_P$ for the parent's mean (writing $\bar{y}_P$ rather than $\bar{y}_R$ to avoid confusion with the right child R_R). Then

$$\text{MSE}(R) = w_L \text{MSE}(R_L) + w_R \text{MSE}(R_R) + w_L(\bar{y}_L - \bar{y}_P)^2 + w_R(\bar{y}_R - \bar{y}_P)^2,$$

so that

$$\Delta = \text{MSE}(R) - w_L \text{MSE}(R_L) - w_R \text{MSE}(R_R) = w_L(\bar{y}_L - \bar{y}_P)^2 + w_R(\bar{y}_R - \bar{y}_P)^2 \geq 0,$$

with equality if and only if $\bar{y}_L = \bar{y}_R = \bar{y}_P$.

In both cases, the proposition confirms each split either reduces impurity or leaves it unchanged – but not that the greedy sequence of splits minimises the impurity of the final tree: a split that helps little now may enable large reductions one level deeper, and greedy search can foreclose those gains.

Example 2.4. A node with 10 observations (6 class 0, 4 class 1) has $G(R) = 1 - (0.36 + 0.16) = 0.48$. A candidate split gives R_L with 5 class-0, 1 class-1 ($G(R_L) = 10/36 \approx 0.278$) and R_R with 1 class-0, 3 class-1 ($G(R_R) = 6/16 = 0.375$). The impurity reduction is $\Delta = 0.48 - 0.6(0.278) - 0.4(0.375) = 0.163$.

3 Stopping, Pruning, and the Bias–Variance Trade-off

Recursive splitting continues until a stopping criterion is met: maximum depth $d_{\max}$, minimum samples per leaf $n_{\min}$, or minimum impurity reduction δ . A tree grown without constraint, when the training observations can be separated exactly by the available features, achieves zero training error but can memorise noise along with signal – typically very low training-set bias and high flexibility, and correspondingly high variance, the tree analogue of k -NN with $k = 1$ from Lecture 5. (Zero training error is not guaranteed in general: two observations with identical feature vectors but different labels cannot be separated by any sequence of axis-aligned splits, and the resulting leaf cannot be made pure. Zero training error also does not by itself imply zero statistical bias relative to f^* ; it describes only the fit to the observed sample.) Restricting depth trades this variance for bias, in a manner broadly analogous to restricting k in k -NN.

Post-pruning grows a large tree $T_{\max}$ and then removes branches, observing the full tree before deciding what to cut – avoiding the myopia of stopping early.

Definition 3.1. For $\alpha \geq 0$, the **cost-complexity** of a subtree $T \subseteq T_{\max}$ is $R_\alpha(T) = R(T) + \alpha|T|$, where

$$R(T) = \sum_{\ell \in \mathcal{L}(T)} \frac{n_\ell}{n} I(R_\ell)$$

is the size-weighted sum of leaf impurities over the leaves $\mathcal{L}(T)$ of T (with n_ℓ the number of training points in leaf ℓ), and $|T|$ is the number of leaves. **Cost-complexity pruning** finds, for each α , the subtree minimising R_α . As α increases from 0, this generates a nested sequence $T_{\max} = T_0 \supseteq T_{\alpha_1} \supseteq \dots \supseteq \{\text{root}\}$, and cross-validation selects the best α from this sequence.

Example 3.2. Three candidate subtrees: T_1 (5 leaves, $R = 0.10$), T_2 (3 leaves, $R = 0.14$), T_3 (1 leaf, $R = 0.22$). Penalised costs are $R_\alpha(T_1) = 0.10 + 5\alpha$, $R_\alpha(T_2) = 0.14 + 3\alpha$, $R_\alpha(T_3) = 0.22 + \alpha$. Crossovers occur at $\alpha = 0.02$ (T_1 to T_2) and $\alpha = 0.04$ (T_2 to T_3): T_1 is uniquely optimal for $0 \leq \alpha < 0.02$; T_1 and T_2 tie at $\alpha = 0.02$; T_2 is uniquely optimal for $0.02 < \alpha < 0.04$; T_2 and T_3 tie at $\alpha = 0.04$; and T_3 is uniquely optimal for $\alpha > 0.04$.

4 Feature Importance

Definition 4.1. The **mean decrease in impurity** (MDI) of feature j in a fitted tree T is

$$\text{MDI}(j) = \sum_{t: j_t=j} \frac{n_t}{n} \Delta(j, \theta_t),$$

summed over internal nodes where j is used to split, with n_t the number of training points reaching node t . Summed over all features, MDI telescopes to $I_{\text{root}} - I_{\text{leaves}}$, the total impurity reduction from root to leaves.

Remark 4.2. MDI has a well-documented bias: features with many distinct values have more candidate split points and are more likely to be selected by the greedy algorithm, even without being genuinely more predictive. An alternative less affected by this split-selection bias is **permutation importance**: after fitting, permute feature j 's values on a held-out set and record the increase in test error. Genuinely predictive features cause a large increase when permuted; irrelevant features cause little change. Both measures are examined in the Workshop, and both extend naturally to the ensembles of Section 5.

The instability noted at the start of this lecture is not merely an inconvenience. A small change in the training data – replacing one point, correcting one label – can change the *first* split of a tree, cascading into an entirely different structure below it. If B trees are trained on B resamples of $\mathcal{D}$ and their predictions averaged, each tree individually has high variance but the average has much lower variance, provided the trees are not too correlated. For identically distributed estimators with variance σ^2 and pairwise correlation ρ ,

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x})\right) = \rho\sigma^2 + \frac{1-\rho}{B} \sigma^2, \quad (1)$$

which converges to $\rho\sigma^2$ as $B \rightarrow \infty$. Averaging reduces variance relative to a single learner whenever $\rho < 1$ (the $B = 1$ case gives $\sigma^2 > \rho\sigma^2 + (1-\rho)\sigma^2/B$ precisely when $\rho < 1$; at $\rho = 1$ there is no reduction), and reducing ρ lowers the limiting variance $\rho\sigma^2$ further. These two levers – averaging, and decorrelating – are exactly bagging and Random Forests respectively.

5 Ensemble Methods

5.1 Bagging

Definition 5.1. Bootstrap aggregation (bagging) draws B bootstrap resamples $\mathcal{D}^{(1)}, \dots, \mathcal{D}^{(B)}$ (each of size n , sampled with replacement from $\mathcal{D}$), fits $\hat{f}^{(b)}$ on each, and averages: $\hat{f}_{\text{bag}} = B^{-1} \sum_b \hat{f}^{(b)}$ for regression, or a majority vote for classification.

Bagging helps most when the base learner is *unstable* – trees are the canonical example; bagging a stable learner such as linear regression yields little improvement, since σ^2 is already small. Bagging is primarily targeted at reducing variance rather than bias. Conditional on the observed training set $\mathcal{D}$, each bootstrap sample $\mathcal{D}^{(b)}$ is drawn independently, with replacement, from the empirical distribution $\hat{P}_n$ placing mass $1/n$ on each observed point – not, in general, an i.i.d. sample from the true underlying distribution P – so averaging over bootstrap resamples changes the variance of $\hat{f}_{\text{bag}}$ relative to a single fitted model without a general guarantee about how it changes the model’s bias relative to f^* . What bagging reliably does is stabilise the predictions of an unstable learner by averaging over many resamples of the one observed dataset.

Remark 5.2 (Out-of-bag error). Each bootstrap resample omits approximately 37% of the training points, since the probability a given point is excluded from a size- n resample is $(1 - 1/n)^n \rightarrow e^{-1}$. These **out-of-bag** (OOB) points were not used to fit $\hat{f}^{(b)}$ and so provide a built-in estimate of generalisation error without requiring a separate validation set: for each i , average the predictions of only the trees for which i was OOB, and compute the empirical risk of these predictions over all i . This estimate is a useful and convenient internal diagnostic and often tracks held-out test performance closely in practice, though it is not generally an unbiased estimate of generalisation error.

5.2 Random Forests

If a small number of features dominate every tree, bagged trees remain highly correlated (ρ close to 1) and averaging provides little benefit in the limit, since $\text{Var} \rightarrow \rho\sigma^2$ regardless of B .

Definition 5.3. A **Random Forest** modifies recursive binary splitting so that, at each node, only a random subset of $m \leq p$ features (drawn afresh at every node) is considered as candidates for the split. Classical rules of thumb include $m \approx \sqrt{p}$ for classification and $m \approx p/3$ for regression; modern software implementations may use different defaults, so m should be regarded as a tunable hyperparameter rather than a fixed prescription. Setting $m = p$ recovers bagging.

Restricting the candidate features at each split forces different trees to use different features near the root, reducing ρ below the bagged value – at some cost to each individual tree’s strength (σ^2 rises slightly). The reduction in ρ typically dominates, giving a net reduction in ensemble variance. Smaller m decorrelates further but weakens individual trees more; the choice of m can be tuned using OOB error or cross-validation. Feature importance extends to a forest by averaging MDI (or permutation importance, computed on OOB observations) across all B trees, which stabilises the single-tree estimates of Section 4.

5.3 Gradient Boosting

Bagging and Random Forests are primarily motivated as variance-reduction methods, averaging weakly correlated models. Gradient boosting instead builds models sequentially, each correcting the errors of its predecessors, and can substantially reduce bias; in practice its effect on variance also depends on the tree depth, learning rate, and number of boosting rounds used.

Definition 5.4. **Gradient boosting** for regression with squared loss initialises $\hat{f}^{(0)} = \bar{y}$, then for $m = 1, \dots, M$: computes pseudo-residuals $r_i^{(m)} = y_i - \hat{f}^{(m-1)}(\mathbf{x}_i)$, fits a shallow tree $T^{(m)}$ to $\{(\mathbf{x}_i, r_i^{(m)})\}$, and updates $\hat{f}^{(m)} = \hat{f}^{(m-1)} + \eta T^{(m)}$ for learning rate $\eta \in (0, 1]$.

Since $\partial \ell(y, \hat{f}) / \partial \hat{f} = \hat{f} - y$ for squared loss $\ell(y, \hat{f}) = (y - \hat{f})^2 / 2$, the pseudo-residual $r_i^{(m)}$ is exactly the negative gradient of the empirical risk with respect to the current prediction: gradient boosting

is gradient descent in *function space*, with a tree as the step direction at each iteration (Friedman, 2001). This view extends to any differentiable loss by substituting the appropriate negative gradient – for cross-entropy loss in binary classification, $r_i^{(m)} = y_i - \sigma(\hat{f}^{(m-1)}(\mathbf{x}_i))$.

Three hyperparameters govern the trade-off: the number of trees M , the learning rate η , and the tree depth d . Unlike a Random Forest, whose generalisation error typically stabilises rather than deteriorates as B grows, gradient boosting *can* overfit as M increases if η is not small enough; early stopping (monitoring validation error and halting when it starts to rise) or cross-validated M is standard practice. Small η requires more trees but typically generalises better, so η and M are tuned jointly – $\eta \leq 0.1$ with early stopping is common. Shallow trees ($d \in \{2, 3, 4\}$) are standard, since the sequential structure lets complex functions emerge from many simple trees.

Remark 5.5 (Modern implementations). XGBoost and LightGBM extend Definition 5.4 with second-order loss approximations and ℓ_1/ℓ_2 regularisation on leaf weights. LightGBM additionally introduces techniques such as gradient-based one-side sampling (which retains observations with large gradients and subsamples among those with small gradients, while correcting for the resulting bias in the gradient estimate) and exclusive feature bundling (which merges sparse, mutually exclusive features into a single feature) to improve computational efficiency on large, high-dimensional datasets. Both expose the same core hyperparameters (M, η, d) through scikit-learn-compatible interfaces.

5.4 Random Forests versus Gradient Boosting

Random Forests are robust and require little tuning: increasing B typically stabilises rather than worsens generalisation performance, they are trivially parallelisable, and the OOB error gives a convenient estimate of generalisation without a separate validation set. Gradient boosting typically achieves lower error when carefully tuned, particularly on structured tabular data, but requires joint tuning of (M, η, d) , is sequential rather than parallel, and is more sensitive to noise and outliers, since large residuals are amplified by the next tree fitted to them. A reasonable default: start with a Random Forest, and move to gradient boosting only if further tuning effort is justified.

6 Exercises

Exercise 6.1. A classification node contains 12 observations: 8 of class A, 4 of class B. A proposed split gives a left child with 7 class-A and 1 class-B, and a right child with 1 class-A and 3 class-B. Compute the Gini impurity of the parent and each child, and the impurity reduction Δ . State the plurality-class prediction at each node.

Solution. Parent: $\hat{p}_A = 2/3$, $G = 1 - (4/9 + 1/9) = 4/9 \approx 0.444$. Left ($n_L = 8$): $\hat{p}_A = 7/8$, $G(R_L) = 1 - (49/64 + 1/64) = 7/32 \approx 0.219$. Right ($n_R = 4$): $\hat{p}_A = 1/4$, $G(R_R) = 1 - (1/16 + 9/16) = 3/8 = 0.375$.

$$\Delta = \frac{4}{9} - \frac{8}{12} \cdot \frac{7}{32} - \frac{4}{12} \cdot \frac{3}{8} = \frac{64}{144} - \frac{21}{144} - \frac{18}{144} = \frac{25}{144} \approx 0.174.$$

Plurality predictions: parent class A ($8 > 4$), left child class A ($7 > 1$), right child class B ($3 > 1$). $\square$

Exercise 6.2. Let $\hat{f}_1, \dots, \hat{f}_B$ be identically distributed with $\text{Var}(\hat{f}_b) = \sigma^2$ and $\text{Corr}(\hat{f}_b, \hat{f}_{b'}) = \rho$ for $b \neq b'$. Derive equation (1), and use it to explain why a Random Forest with $\rho = 0.2$, $\sigma^2 = 0.05$ has lower limiting variance than bagging with $\rho = 0.6$, $\sigma^2 = 0.04$, despite the individual Random Forest trees being weaker.

Solution. Let $\bar{f} = B^{-1} \sum_b \hat{f}_b$. Then

$$\text{Var}(\bar{f}) = \frac{1}{B^2} [B\sigma^2 + B(B-1)\rho\sigma^2] = \frac{\sigma^2}{B} + \frac{(B-1)\rho\sigma^2}{B} = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{B},$$

using $\text{Cov}(\hat{f}_b, \hat{f}_{b'}) = \rho\sigma^2$ for each of the $B(B-1)$ pairs. As $B \rightarrow \infty$ this converges to $\rho\sigma^2$.

Bagging: limiting variance = $0.6 \times 0.04 = 0.024$. Random Forest: limiting variance = $0.2 \times 0.05 = 0.010$.

Despite σ^2 rising from 0.04 to 0.05 (individual trees weaker under feature subsampling), the drop in ρ from 0.6 to 0.2 dominates, giving the Random Forest a limiting variance $2.4\times$ lower than bagging. In this example, the benefit from decorrelating the trees outweighs the increase in individual-tree variance. $\square$

Exercise 6.3. Let T be a classification tree on $n = 100$ points with the following internal-node structure (feature, impurity reduction Δ , points reaching the node n_t):

Node	Depth	Feature	Δ	n_t
Root	0	x_1	0.180	100
Left child of root	1	x_3	0.090	60
Right child of root	1	x_1	0.060	40
Left-left grandchild	2	x_2	0.040	35
Left-right grandchild	2	x_4	0.020	25
Right-left grandchild	2	x_3	0.050	22
Right-right grandchild	2	x_1	0.030	18

Compute the unnormalised MDI for each feature, verify the total equals the sum of all weighted impurity reductions, and rank the features by normalised importance.

Solution.

$$\text{MDI}(x_1) = \frac{100}{100}(0.180) + \frac{40}{100}(0.060) + \frac{18}{100}(0.030) = 0.180 + 0.024 + 0.0054 = 0.2094.$$

$$\text{MDI}(x_2) = \frac{35}{100}(0.040) = 0.0140, \quad \text{MDI}(x_3) = \frac{60}{100}(0.090) + \frac{22}{100}(0.050) = 0.0650,$$

$$\text{MDI}(x_4) = \frac{25}{100}(0.020) = 0.0050.$$

Total = $0.2094 + 0.0140 + 0.0650 + 0.0050 = 0.2934$, matching the direct sum of all seven weighted node reductions ($0.180 + 0.054 + 0.024 + 0.014 + 0.005 + 0.011 + 0.0054 = 0.2934$).

Normalised: $x_1 \approx 0.714$, $x_3 \approx 0.222$, $x_2 \approx 0.048$, $x_4 \approx 0.017$. Ranking: $x_1 \gg x_3 > x_2 > x_4$, with x_1 dominant because it appears at the root (governing all 100 points) and twice more, deeper in the tree. $\square$

Exercise 6.4. A gradient boosting model with squared loss is initialised at $\hat{f}^{(0)} = \bar{y}$ and trained for one step with learning rate $\eta = 0.5$ on four observations:

i	x_i	y_i	$\hat{f}^{(0)}(x_i)$
1	1	3	2.5
2	2	4	2.5
3	3	1	2.5
4	4	2	2.5

A depth-1 tree $T^{(1)}$ fitted to the residuals predicts $T^{(1)}(x) = 1.0$ for $x \leq 2.5$ and -1.0 for $x > 2.5$. Compute the residuals, the updated predictions $\hat{f}^{(1)}(x_i)$, and the training MSE before and after the update.

Solution. Residuals: $r_i^{(1)} = y_i - 2.5 = (0.5, 1.5, -1.5, -0.5)$.

Updated predictions: for $i = 1, 2$ ($x \leq 2.5$), $\hat{f}^{(1)} = 2.5 + 0.5(1.0) = 3.0$; for $i = 3, 4$ ($x > 2.5$), $\hat{f}^{(1)} = 2.5 + 0.5(-1.0) = 2.0$.

Initial MSE: $\frac{1}{4}[(0.5)^2 + (1.5)^2 + (-1.5)^2 + (-0.5)^2] = \frac{1}{4}(0.25 + 2.25 + 2.25 + 0.25) = 1.25$.

MSE after step 1: $\frac{1}{4}[(3 - 3)^2 + (4 - 3)^2 + (1 - 2)^2 + (2 - 2)^2] = \frac{1}{4}(0 + 1 + 1 + 0) = 0.5$.

The MSE has fallen from 1.25 to 0.5 after a single boosting step. $\square$

Further Reading

Breiman, Friedman, Olshen, and Stone (1984) is the original CART monograph, covering recursive binary splitting, Gini impurity, and cost-complexity pruning in full. Quinlan (1986) introduces the entropy-based ID3 algorithm as an alternative to CART. Breiman (1996, 2001) are the original papers on bagging and Random Forests respectively; Friedman (2001) introduces the functional gradient descent view of boosting used in Section 5.3. Chen and Guestrin (2016) and Ke et al. (2017) describe XGBoost and LightGBM. Lindholm et al. and James et al. both treat trees and ensembles as a unified family, at a level consistent with this lecture; Hastie, Tibshirani, and Friedman (2009), Chapters 9 – 10 and 15, is the standard graduate-level reference for both topics.

References

- L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone (1984). *Classification and Regression Trees*. Wadsworth.
- J. R. Quinlan (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.
- L. Breiman (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140.
- L. Breiman (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- J. H. Friedman (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- T. Chen and C. Guestrin (2016). XGBoost: a scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu (2017). LightGBM: a highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154.
- A. Lindholm, N. Wahlström, F. Lindsten, and T. B. Schön (2022). *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press.
- G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer.

T. Hastie, R. Tibshirani, and J. Friedman (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.